Project 3: Real-time 2-D Object Recognition

Team Members

- **Sangeeth Deleep Menon** | NUID: 002524579 | MSCS - Boston | CS5330 Section 03 (CRN: 40669, Online)
- **Raj Gupta** | NUID: 002068701 | MSCS - Boston | CS5330 Section 01 (CRN: 38745, Online)

Project Description

This project implements a real-time 2-D object recognition system that identifies objects placed on a white surface in a translation-, scale-, and rotation-invariant manner. The system supports webcam, single image, image directory, and video input modes. All four core pipeline stages: thresholding, morphological filtering, connected components analysis, and moment/orientation computation are written **from scratch** without using OpenCV built-in functions. Classification is performed using hand-crafted features (Hu moment invariants), eigenspace (PCA) embeddings, and CNN (ResNet18) embeddings.

Building the Project

This project uses CMake and requires OpenCV (with `dnn` module) and optionally Qt for the GUI. A C++17 compatible compiler is required.

1. **Navigate to the project directory.**
2. **Create a build directory and run CMake and make:**

```
mkdir -p cmake-build-debug && cd cmake-build-debug
cmake ..
make
```

This will create two executables inside the `cmake-build-debug` directory: `Project3` (command-line) and `Project3_GUI` (interactive GUI, requires Qt).

Running the Applications

Command-Line Application

The executable supports several input modes:

```
# Webcam mode (default)
./cmake-build-debug/Project3

# Single image
./cmake-build-debug/Project3 -i images/cable.jpg

# Directory of images
./cmake-build-debug/Project3 -d images/
```

```
# Video file
./cmake-build-debug/Project3 -v images/pen.mp4
```

GUI Application

The GUI provides a more user-friendly way to use the system.

1. **Navigate to the project's root directory.**
2. **Execute the command:**

   ```
   ./cmake-build-debug/Project3_GUI
   ```

3. **How to Use:**
   - Click **"Webcam"**, **"Open Image..."**, **"Open Directory..."**, or **"Open Video..."** to select an input source.
   - All four pipeline stages (thresholded, cleaned, regions, overlays) are displayed in tiled panels.
   - Select the classification method from the **dropdown** (Hand-crafted Features / Eigenspace / CNN).
   - Adjust pipeline parameters (threshold bias, morphology radii, min area, blur) using **sliders and spinboxes** — changes update in real time.
   - Click **"Train Features"**, **"Train CNN Embed"**, or **"Save ROI"** to add training data.
   - Click **"Build Eigenspace"** to construct the PCA space from saved ROIs.
   - Use **"Ground Truth"** to evaluate predictions and **"Print Confusion Matrix"** to view results.
   - A timestamped log panel at the bottom tracks all actions.

Key Bindings (Command-Line Application)

Key	Action
`q`	Quit
`0`	Toggle classification ON/OFF
`1` / `2` / `3`	Classify with: Features / Eigenspace / CNN
`4`	Train — save hand-crafted features with a label
`5`	Save preprocessed ROI for eigenspace training
`6`	Build eigenspace from saved ROIs
`7`	Save CNN embedding for current object
`8`	Ground-truth evaluation (for confusion matrix)
`9`	Print confusion matrix
`s`	Save screenshot
`+` / `=`	Increase threshold bias

Key	Action
`-`	Decrease threshold bias
`space`	Pause / resume
`.` / `,`	Next / previous image (directory mode)

Executable Files

This project generates two main executable files, each with a specific role:

1. `Project3` **(Command-Line Interface - CLI)**
    - **Purpose**: The core application run directly from the terminal. It supports webcam, single image, directory, and video modes. Training, classification, and evaluation are controlled via key presses.
2. `Project3_GUI` **(Graphical User Interface - GUI)**
    - **Purpose**: The interactive application with a full Qt desktop window. It provides buttons, dropdowns, sliders, and dialog boxes for all pipeline operations — no keyboard shortcuts needed. Includes a dark theme and live parameter adjustment.

Methods Overview

Task	Stage	Implementation
1. Thresholding	HSV saturation+value scoring	**From scratch** — ISODATA (K=2 k-means) adaptive threshold
2. Morphological Filtering	Closing → Opening cleanup	**From scratch** — Erosion/dilation with circular structuring element
3. Connected Components	Region segmentation	**From scratch** — Two-pass algorithm with union-find (8-connectivity)
4. Feature Computation	Moments, orientation, Hu invariants	**From scratch** — Raw/central/normalized moments, oriented bounding box
5–6. Training & Classification	Scaled Euclidean nearest-neighbour	CSV-based object DB with per-feature std dev normalization
7. Confusion Matrix	Performance evaluation	8x8 matrix with accuracy reporting
9a. Eigenspace Embedding	PCA one-shot classification	SVD-based eigenspace build, project, SSD nearest-neighbour
9b. CNN Embedding	ResNet18 one-shot classification	ONNX model embedding, SSD nearest-neighbour

Project Files

File	Description
`main.cpp`	CLI main loop, key handling, display
`main_gui.cpp`	Qt application entry point with dark theme
`gui.cpp` / `gui.h`	Qt GUI — panels, controls, parameter sliders, training dialogs
`thresholding.cpp` / `thresholding.h`	**From scratch**: ISODATA adaptive thresholding
`morphological.cpp` / `morphological.h`	**From scratch**: erosion, dilation, opening, closing
`segmentation.cpp` / `segmentation.h`	**From scratch**: two-pass connected components with union-find
`features.cpp` / `features.h`	**From scratch**: moments, orientation, Hu moment invariants, oriented BB
`classifier.cpp` / `classifier.h`	Scaled Euclidean nearest-neighbour classifier, confusion matrix
`eigenspace.cpp` / `eigenspace.h`	PCA eigenspace build, project, classify, save/load
`utilities.cpp`	CNN embedding via ResNet18 — provided by instructor (Bruce Maxwell)
`vision.h`	Common structs (RegionStats, RegionFeatures, TrainingEntry)
`CMakeLists.txt`	Build configuration for both CLI and GUI targets

Extensions

- A full Qt desktop GUI was developed with tiled pipeline visualization, dropdown classification method selection, live parameter sliders, training/evaluation dialogs, and a timestamped log panel. The GUI provides a dark-themed, professional interface for all pipeline operations.
- All four core pipeline stages (thresholding, morphological filtering, connected components, and moment/orientation computation) were written from scratch without using OpenCV built-in functions.
- Both eigenspace (PCA) and CNN (ResNet18) embedding methods were implemented for one-shot classification (Task 9), with comparative evaluation.

Time Travel Days

3 days used.

Videos

Google Drive Link: https://drive.google.com/file/d/1W-ve_jZqCAEpJYs-T56vFMFhPxUU_AvP/view?usp=sharing

Panopto Link: https://northeastern.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=cd177581-4d19-4323-82dd-b3fe003270ac

Acknowledgements

- OpenCV documentation for image I/O and DNN module references
- Course materials and sample code provided by Prof. Bruce Maxwell (utilities.cpp, embedding.py, ResNet18 ONNX model)
- An AI assistant (Claude) was used to help write and debug code, and for project documentation.